

Paper Retrieval Features: Multimodal Embeddings, Relevant Paper Search, and Citation Graphs

Imran Kutianawala
Kellis Lab, MIT

July 2026

This report covers three features built for the paper retrieval side of Mantis. The first is a multimodal embedding pipeline built on Gemini 2 Embedding that turns arbitrary user uploads into vectors. The second is a RAG system that uses those vectors to surface relevant research papers from a large scraped corpus. The third is a citation graph generator that takes a seed paper and pulls the papers it cites through the OpenAlex API. The three pieces share tools where it made sense, so the embedding work in Section 1 is used in the retrieval in Section 2.

1 Multimodal Embeddings with Gemini 2 Embedding

We had previously discussed Gemini 2 Embedding as the model for multimodal embeddings, since it lets users upload diverse documents and supports advanced RAG over new data types. This week I implemented it end to end and wrote a set of tests to make sure it holds up at a production level rather than just on clean demo inputs.

1.1 Setup

The pipeline runs on the `google-genai` client against `gemini-embedding-2`, with output dimensionality set to 1536 so the input vectors and the corpus vectors from Section 2 live in the same space. User uploads are embedded with a query template and corpus documents with a document template, which is the asymmetric setup the model expects for retrieval. The client reads the API key from the environment, and every run writes two things: a JSONL file with one record per embedded chunk (source file, content hash, chunk index, metadata, and the vector) and a manifest that reports per-file status, so it is always clear which files embedded fully, which partially, and which were skipped and why.

1.2 File-type handling

I didn't want to trust file extensions, since users rename things all the time. Each file is identified by sniffing its magic bytes first, which covers PDF, PNG, JPEG, WAV, WebP, Ogg, FLAC, MP3, AAC, WebM, MPEG, and the whole ISO media family. That last group is the annoying one because HEIC photos, M4A audio, and MP4 video all share the same container header, so the pipeline reads the container brand and, when that is ambiguous, probes the streams with `ffmpeg` to decide whether a file is actually audio or video. DOCX files are detected as zip archives containing a Word document and converted to PDF before chunking. Only when sniffing fails does the extension get

consulted, and a file claiming to be text still has to pass a printability check on its first bytes before it is treated as text. Anything that fails all of this is reported as unsupported instead of being sent to the API to fail there.

1.3 Chunking

Gemini limits how much content fits in a single embedding request, so I built a chunking system for the five media types we support, each with overlap between consecutive chunks so content near a boundary is never lost:

- **Text** (plain, HTML, CSV, RTF) is stripped to plain text and split into 3,500 word windows with 300 words of overlap, plus a character cap as a backstop for text with abnormally long tokens.
- **PDFs** (and converted DOCX) are split into two-page windows with one page of overlap, written out as valid standalone PDFs per chunk.
- **Images** (JPEG, PNG, WebP, HEIC/HEIF) are single chunks. Oversized or exotic formats get downsampled to 2048px and transcoded to PNG, and images are batched three per request since they are cheap token-wise.
- **Video** is cut into 60 second segments with 20 seconds of overlap and re-encoded to a small 640px H.264 stream so each segment stays under the inline upload limit.
- **Audio** is cut into 30 second segments with 15 seconds of overlap and downmixed to 16kHz mono WAV.

1.4 Rate limiting and error handling

Gemini has strict rate limits on both requests and tokens per minute, so I wrote a sliding-window rate limiter that tracks both budgets at once. Each chunk gets a token estimate based on its media type (characters for text, pages for PDFs, seconds for audio and video, a flat cost for images), and the limiter sleeps just long enough to stay under 90 requests and 28,000 tokens per minute. Both caps are configurable through environment variables for when we move to a higher tier.

On top of that, transient API failures (429s, timeouts, 5xx errors) are retried up to five times with exponential backoff. If a batched image request fails, the pipeline falls back to embedding each image individually so one bad file can't take down the rest of its batch. Failures that are the file's fault, like password-protected PDFs, corrupt media, or empty files, are caught per file, logged with a reason, and recorded in the manifest without stopping the run.

1.5 Testing

I created more than five test cases to stress the file handling before calling it done. These included uploading random file types that do not exist, files with lying extensions (for example media renamed to `.txt`), a batch of over 20 mixed files in one run, empty files, and long documents that force the chunker through many windows, alongside normal PDFs and images to confirm the happy path. The main things these caught early were the HEIC versus M4A versus MP4 container ambiguity and the need for the per-file failure isolation described above. In every test the run completes, every file ends up in the manifest as embedded, partially embedded, or skipped with a stated reason, and no single bad input can crash the pipeline.

2 RAG for Relevant Paper Finding

The purpose of this feature is that when a user uploads text or documents, we fetch on the order of 5,000 relevant research papers for them to explore. The embedding pipeline from Section 1 handles the user side, so the work here was building the corpus and the retrieval logic.

2.1 Building the corpus

No single source had the coverage we wanted, so I scraped from multiple places, primarily Kaggle datasets of paper metadata with Google Patents as a secondary source, ending up with over 2.5 million research papers. The raw pull needed a real cleaning pass before it was usable: deduplicating papers that appeared across sources, dropping entries with missing or truncated abstracts, and normalizing everything into one schema of id, title, authors, categories, and abstract.

2.2 Retrieval methodology

Each corpus paper is embedded once as `title + abstract` using the document template, through the same rate-limited, retry-wrapped Gemini client as Section 1, and stored in the same JSONL record format so the input and corpus sides are interchangeable downstream. Retrieval is cosine similarity over the normalized 1536-dimensional vectors, but with one wrinkle for chunked inputs. A long upload becomes many chunks, and averaging all of them washes out the signal from the sections that actually matter. So for each paper I take the best-matching input chunk scores and average only the top 20 percent of them (with a floor of two chunks). This rewards papers that strongly match some part of the upload without requiring them to match all of it, which is the right behavior for something like a thesis where only one chapter is relevant to a given paper.

Since embedding 2.5 million abstracts is expensive, I validated the RAG functionality first on a sample of roughly 100 arXiv papers embedded with Gemini 2 Embedding. On that testbed, uploads reliably rank their genuinely related papers at the top, including cross-modal cases where the input is an image or a PDF rather than plain text. The corpus embedding job is written to scale to the full dataset as the next step, since it already handles rate pacing, retries, and skip-and-continue on bad records.

3 Citation Graph Generation

This feature works like relevant papers, but instead of surfacing related work by similarity, it takes a seed paper and retrieves the papers it cites, giving the user the actual reference structure around a paper rather than a semantic neighborhood.

3.1 OpenAlex setup

I built this on the OpenAlex API. The entry point accepts whatever the user is likely to paste: an arXiv ID or link, a DOI, an OpenAlex work ID, or a plain title search, and each form resolves to a work record through the appropriate endpoint (arXiv IDs, for instance, resolve through their landing page URL, with a DOI-based fallback since OpenAlex indexes some arXiv papers only one way). From the seed, the graph is grown breadth-first: each expansion fetches both the works a paper references and the works that cite it, interleaves the two lists so neither direction dominates, and adds new nodes tagged with their depth and whether they entered as a reference or a citation, up to a configurable paper cap.

Every returned work is normalized into the schema the rest of the app uses, which took some cleanup on the OpenAlex side. Abstracts come back as inverted indexes and get reconstructed into text, DOIs and arXiv IDs are pulled out of several possible fields, and publication years get sanity-checked against the arXiv ID (OpenAlex occasionally reports a future year for a paper, in which case the year encoded in the arXiv ID wins).

3.2 Rate limiter handling

OpenAlex asks clients to identify themselves and stay around ten requests per second, so every request carries a `mailto` parameter and identifying user agent, which puts us in their polite pool with better rate limits. The client enforces a minimum gap between consecutive requests to stay under the cap proactively, and when a 429 comes back anyway it retries with exponential backoff. Responses are cached in-process, which matters here because a breadth-first expansion revisits the same works often, and paginated fetches request only the fields we actually use to keep responses small. Failures are turned into readable errors, so a bad identifier tells the user to try a DOI or arXiv ID instead of surfacing a raw HTTP error. In testing, a 200-paper graph builds in well under a minute without ever tripping the rate limit.

4 Summary and Next Steps

The common thread across all three features is assuming that inputs are messy and APIs are flaky, and making every failure isolated and explained. The embedding pipeline survives fake and mislabeled files, the corpus job skips bad records and keeps going, and the citation crawler paces itself and retries. Next up is scaling the corpus embedding from the 100-paper testbed toward the full 2.5 million papers and moving retrieval into the database layer so the 5,000-paper fetch runs at interactive speed.